Linear Transformations and Vector Operations for Machine Learning and Deep Learning

Akshay Redekar

December 23, 2025

Contents

1 Linear Transformation of Non-square Matrix **3**
1.1 Introduction and Motivation . . . 3
1.2 Mathematical Properties . . . 3
1.3 Types of Non-square Transformations . . . 4
1.3.1 Expansion: From Lower to Higher Dimension . . . 4
1.3.2 Reduction: From Higher to Lower Dimension . . . 4
1.4 Left and Right Inverses . . . 5
1.5 Applications in Machine Learning . . . 6

2 Dot Product **10**
2.1 Definition and Mathematical Foundation . . . 10
2.2 Geometric Interpretation . . . 10
2.3 Computational Examples . . . 11
2.4 Applications in Machine Learning and Deep Learning . . . 12

3 Cross Product **17**
3.1 Definition and Mathematical Foundation . . . 17
3.2 Geometric Interpretation . . . 17
3.3 Computational Examples . . . 18
3.4 Applications in Machine Learning and Computer Vision . . . 20
3.5 Cross Product vs Dot Product Summary . . . 23
3.6 Why Cross Product is Less Common in ML . . . 23

4 Conclusion **25**
4.1 Key Takeaways . . . 25
4.2 Practical Recommendations . . . 25
4.3 Further Study . . . 26
4.4 Connection to Previous Topics . . . 26

5 Assessment Questions **27**
5.1 Level 1: Basic Understanding 27
5.2 Level 2: Intermediate 27
5.3 Level 3: Advanced 28
5.4 Multiple Choice Questions 29
5.5 Programming Exercises 29

6 Solutions to Assessment Questions **31**
6.1 Level 1 Solutions 31
6.2 Level 2 Solutions 32
6.3 Level 3 Solutions 34
6.4 Multiple Choice Solutions 37

7 Practice Problems **38**
7.1 Computational Problems 38
7.2 Conceptual Problems 38
7.3 Application Problems 38

1 Linear Transformation of Non-square Matrix

1.1 Introduction and Motivation

In machine learning and deep learning, we frequently encounter non-square matrices because the number of features doesn't usually match the number of data points. Understanding how these matrices transform vectors is fundamental to many ML algorithms.

Definition 1.1 (Non-square Matrix)**.** A non-square matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ where $m \neq n$ represents a linear transformation that maps vectors from $\mathbb{R}^n$ to $\mathbb{R}^m$:

$$\mathbf{A} : \mathbb{R}^n \to \mathbb{R}^m \tag{1}$$

- If $m > n$: The transformation maps from lower to higher dimension (expansion)
- If $m < n$: The transformation maps from higher to lower dimension (reduction)

Why Non-square Matrices in ML/DL?

Common Scenarios:

1. **Dataset Matrix:** $\mathbf{X} \in \mathbb{R}^{n \times d}$ where n samples $\neq d$ features
2. **Neural Network Layers:** Input dimension often differs from output dimension
3. **Dimensionality Reduction:** PCA, autoencoders map from high to low dimensions
4. **Feature Extraction:** Convolutional layers change spatial dimensions
5. **Embedding Layers:** Map discrete tokens to continuous vectors (different dimensions)

1.2 Mathematical Properties

Theorem 1.1 (Properties of Non-square Linear Transformations)**.** For matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ with $m \neq n$:

1. **Linearity:** $\mathbf{A}(c_1\mathbf{x}_1 + c_2\mathbf{x}_2) = c_1\mathbf{A}\mathbf{x}_1 + c_2\mathbf{A}\mathbf{x}_2$
2. **Rank:** $\text{rank}(\mathbf{A}) \leq \min(m, n)$
3. **Not Invertible:** Cannot have a true inverse since $m \neq n$, but may have:
   - Left inverse if $m > n$ and full column rank
   - Right inverse if $m < n$ and full row rank
   - Pseudo-inverse in general case
4. **Null Space:** If $m < n$, the null space is non-trivial: $\dim(\mathcal{N}(\mathbf{A})) = n - \text{rank}(\mathbf{A}) > 0$

1.3 Types of Non-square Transformations

1.3.1 Expansion: From Lower to Higher Dimension

Example 1.1 (Dimension Expansion $\mathbb{R}^2 \to \mathbb{R}^3$)**.** Consider transformation matrix:

$$\mathbf{A} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix} \in \mathbb{R}^{3\times 2} \tag{2}$$

Step 1: Apply to input vector

$$\mathbf{x} = \begin{bmatrix} 2 \\ 3 \end{bmatrix} \in \mathbb{R}^2 \tag{3}$$

Step 2: Compute transformation

$$\mathbf{y} = \mathbf{A}\mathbf{x} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 2 \\ 3 \end{bmatrix} \tag{4}$$

$$= \begin{bmatrix} 1(2) + 0(3) \\ 0(2) + 1(3) \\ 1(2) + 1(3) \end{bmatrix} = \begin{bmatrix} 2 \\ 3 \\ 5 \end{bmatrix} \in \mathbb{R}^3 \tag{5}$$

Interpretation:

- First component: keeps x_1
- Second component: keeps x_2
- Third component: sum $x_1 + x_2$

Geometric view: The 2D vector is embedded into 3D space along a plane.
Range: All outputs lie in a 2D plane within $\mathbb{R}^3$:

$$\mathcal{C}(\mathbf{A}) = \text{span}\left\{ \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix} \right\} \tag{6}$$

1.3.2 Reduction: From Higher to Lower Dimension

Example 1.2 (Dimension Reduction $\mathbb{R}^3 \to \mathbb{R}^2$)**.** Consider transformation matrix:

$$\mathbf{A} = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix} \in \mathbb{R}^{2\times 3} \tag{7}$$

Step 1: Apply to input vector

$$\mathbf{x} = \begin{bmatrix} 2 \\ 3 \\ 1 \end{bmatrix} \in \mathbb{R}^3 \tag{8}$$

Step 2: Compute transformation

$$\mathbf{y} = \mathbf{A}\mathbf{x} = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix} \begin{bmatrix} 2 \\ 3 \\ 1 \end{bmatrix} \tag{9}$$

$$= \begin{bmatrix} 1(2) + 0(3) + 1(1) \\ 0(2) + 1(3) + 1(1) \end{bmatrix} = \begin{bmatrix} 3 \\ 4 \end{bmatrix} \in \mathbb{R}^2 \tag{10}$$

Interpretation:

- First component: $x_1 + x_3$
- Second component: $x_2 + x_3$
- Information from 3D is projected to 2D

Null Space: Find vectors mapped to zero.
Solve $\mathbf{A}\mathbf{x} = \mathbf{0}$:

$$\begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \tag{11}$$

From equations:

$$x_1 + x_3 = 0 \implies x_1 = -x_3 \tag{12}$$

$$x_2 + x_3 = 0 \implies x_2 = -x_3 \tag{13}$$

Let $x_3 = t$:

$$\mathbf{x} = t \begin{bmatrix} -1 \\ -1 \\ 1 \end{bmatrix} \tag{14}$$

Null space basis:

$$\mathcal{N}(\mathbf{A}) = \text{span} \left\{ \begin{bmatrix} -1 \\ -1 \\ 1 \end{bmatrix} \right\} \tag{15}$$

Dimension analysis:

- $\text{rank}(\mathbf{A}) = 2$
- $\dim(\mathcal{N}(\mathbf{A})) = 3 - 2 = 1$

This shows information loss: any vector along the null space direction is mapped to zero.

1.4 Left and Right Inverses

Definition 1.2 (Left Inverse)**.** For $\mathbf{A} \in \mathbb{R}^{m \times n}$ with $m > n$ and full column rank, the left inverse is:

$$\mathbf{A}_L^{-1} = (\mathbf{A}^T\mathbf{A})^{-1}\mathbf{A}^T \tag{16}$$

Property: $\mathbf{A}_L^{-1}\mathbf{A} = \mathbf{I}_n$ (but $\mathbf{A}\mathbf{A}_L^{-1} \neq \mathbf{I}_m$)

Definition 1.3 (Right Inverse)**.** For $\mathbf{A} \in \mathbb{R}^{m\times n}$ with $m < n$ and full row rank, the right inverse is:

$$\mathbf{A}_R^{-1} = \mathbf{A}^T(\mathbf{A}\mathbf{A}^T)^{-1} \tag{17}$$

Property: $\mathbf{A}\mathbf{A}_R^{-1} = \mathbf{I}_m$ (but $\mathbf{A}_R^{-1}\mathbf{A} \neq \mathbf{I}_n$)

Example 1.3 (Computing Left Inverse)**.** For:

$$\mathbf{A} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix} \in \mathbb{R}^{3\times 2} \tag{18}$$

Step 1: Compute $\mathbf{A}^T\mathbf{A}$

$$\mathbf{A}^T\mathbf{A} = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} \tag{19}$$

Step 2: Invert $\mathbf{A}^T\mathbf{A}$

$$\det(\mathbf{A}^T\mathbf{A}) = 4 - 1 = 3 \tag{20}$$

$$(\mathbf{A}^T\mathbf{A})^{-1} = \frac{1}{3}\begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix} \tag{21}$$

Step 3: Compute left inverse

$$\mathbf{A}_L^{-1} = (\mathbf{A}^T\mathbf{A})^{-1}\mathbf{A}^T \tag{22}$$

$$= \frac{1}{3}\begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix} \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix} \tag{23}$$

$$= \frac{1}{3}\begin{bmatrix} 2 & -1 & 1 \\ -1 & 2 & 1 \end{bmatrix} \tag{24}$$

Verification:

$$\mathbf{A}_L^{-1}\mathbf{A} = \frac{1}{3}\begin{bmatrix} 2 & -1 & 1 \\ -1 & 2 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix} \tag{25}$$

$$= \frac{1}{3}\begin{bmatrix} 3 & 0 \\ 0 & 3 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \mathbf{I}_2 \checkmark \tag{26}$$

1.5 Applications in Machine Learning

ML/DL Applications of Non-square Transformations

1. Dimensionality Reduction (PCA):
Transform high-dimensional data to lower dimensions:

$$\mathbf{Z} = \mathbf{X}\mathbf{W} \quad \text{where } \mathbf{W} \in \mathbb{R}^{d\times k}, k < d \tag{27}$$

Example: Project 1000-dimensional text features to 50 dimensions.

2. Neural Network Layers:

Each layer performs non-square transformation:

$$\mathbf{h}^{(l)} = \sigma(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}) \tag{28}$$

where $\mathbf{W}^{(l)} \in \mathbb{R}^{n_l \times n_{l-1}}$ (often $n_l \neq n_{l-1}$)

3. Feature Engineering:
Create new features through linear combinations:

$$\mathbf{X}_{new} = \mathbf{X}_{old}\mathbf{T} \quad \text{where } \mathbf{T} \in \mathbb{R}^{d_{old} \times d_{new}} \tag{29}$$

4. Autoencoders:

- Encoder: $\mathbf{z} = f(\mathbf{W}_e\mathbf{x})$ where $\mathbf{W}_e \in \mathbb{R}^{k \times d}$, $k < d$
- Decoder: $\hat{\mathbf{x}} = g(\mathbf{W}_d\mathbf{z})$ where $\mathbf{W}_d \in \mathbb{R}^{d \times k}$

5. Word Embeddings (NLP):
Map vocabulary to dense vectors:

$$\mathbf{E} \in \mathbb{R}^{V \times d} \quad \text{where } V \text{ is vocab size, } d \text{ is embedding dim} \tag{30}$$

Typically $V \gg d$ (e.g., $50000 \to 300$)

6. Convolutional Layers:
Flatten spatial dimensions while increasing channels:

$$\text{Input: } 32 \times 32 \times 3 \to \text{Output: } 16 \times 16 \times 64 \tag{31}$$

Example 1.4 (Complete ML Example: PCA Dimensionality Reduction)**.** Dataset with 4 samples, 3 features:

$$\mathbf{X} = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 3 & 5 \\ 3 & 5 & 7 \\ 4 & 6 & 9 \end{bmatrix} \in \mathbb{R}^{4 \times 3} \tag{32}$$

Goal: Reduce from 3 to 2 dimensions.

Step 1: Center the data

$$\boldsymbol{\mu} = \begin{bmatrix} 2.5 \\ 4 \\ 6 \end{bmatrix} \tag{33}$$

$$\mathbf{X}_{centered} = \begin{bmatrix} -1.5 & -2 & -3 \\ -0.5 & -1 & -1 \\ 0.5 & 1 & 1 \\ 1.5 & 2 & 3 \end{bmatrix} \tag{34}$$

Step 2: Compute covariance matrix

$$\mathbf{C} = \frac{1}{3}\mathbf{X}_{centered}^T\mathbf{X}_{centered} = \begin{bmatrix} 1.667 & 2.167 & 3.167 \\ 2.167 & 2.917 & 4.250 \\ 3.167 & 4.250 & 6.667 \end{bmatrix} \tag{35}$$

Step 3: Eigendecomposition

Eigenvalues: $\lambda_1 = 11.01$, $\lambda_2 = 0.24$, $\lambda_3 \approx 0$
Top 2 eigenvectors (principal components):

$$\mathbf{W} = \begin{bmatrix} 0.425 & -0.808 \\ 0.566 & -0.118 \\ 0.706 & 0.578 \end{bmatrix} \in \mathbb{R}^{3\times 2} \tag{36}$$

Step 4: Project data (non-square transformation)

$$\mathbf{Z} = \mathbf{X}_{centered}\mathbf{W} \tag{37}$$

$$= \begin{bmatrix} -1.5 & -2 & -3 \\ -0.5 & -1 & -1 \\ 0.5 & 1 & 1 \\ 1.5 & 2 & 3 \end{bmatrix} \begin{bmatrix} 0.425 & -0.808 \\ 0.566 & -0.118 \\ 0.706 & 0.578 \end{bmatrix} \tag{38}$$

$$\approx \begin{bmatrix} -4.586 & 0.321 \\ -1.531 & 0.069 \\ 1.531 & -0.069 \\ 4.586 & -0.321 \end{bmatrix} \in \mathbb{R}^{4\times 2} \tag{39}$$

Result: Data reduced from 3D to 2D!
Variance explained:

$$\frac{\lambda_1 + \lambda_2}{\lambda_1 + \lambda_2 + \lambda_3} = \frac{11.25}{11.25} \approx 99.8\% \tag{40}$$

Reconstruction (approximate):

$$\mathbf{X}_{reconstructed} = \mathbf{Z}\mathbf{W}^T + \boldsymbol{\mu} \tag{41}$$

This demonstrates how non-square transformation $\mathbf{W} \in \mathbb{R}^{3\times 2}$ reduces dimensionality while preserving 99.8% of variance.

Example 1.5 (Deep Learning Example: Neural Network Layer)**.** Consider a fully connected layer:
Input: $\mathbf{x} \in \mathbb{R}^{784}$ (flattened 28×28 image) **Output:** $\mathbf{h} \in \mathbb{R}^{128}$ (hidden layer)
Transformation:

$$\mathbf{h} = \text{ReLU}(\mathbf{W}\mathbf{x} + \mathbf{b}) \tag{42}$$

where $\mathbf{W} \in \mathbb{R}^{128\times 784}$ (non-square!)
Forward pass example (simplified):
Let's use smaller dimensions for illustration:

$$\mathbf{W} = \begin{bmatrix} 0.1 & 0.2 & 0.3 & 0.1 \\ 0.2 & 0.1 & 0.2 & 0.3 \\ 0.3 & 0.3 & 0.1 & 0.2 \end{bmatrix} \in \mathbb{R}^{3\times 4}, \quad \mathbf{b} = \begin{bmatrix} 0.1 \\ 0.2 \\ 0.1 \end{bmatrix} \tag{43}$$

Input:

$$\mathbf{x} = \begin{bmatrix} 1.0 \\ 2.0 \\ 1.5 \\ 0.5 \end{bmatrix} \in \mathbb{R}^4 \tag{44}$$

Step 1: Linear transformation

$$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b} = \begin{bmatrix} 0.1 & 0.2 & 0.3 & 0.1 \\ 0.2 & 0.1 & 0.2 & 0.3 \\ 0.3 & 0.3 & 0.1 & 0.2 \end{bmatrix} \begin{bmatrix} 1.0 \\ 2.0 \\ 1.5 \\ 0.5 \end{bmatrix} + \begin{bmatrix} 0.1 \\ 0.2 \\ 0.1 \end{bmatrix} \tag{45}$$

$$= \begin{bmatrix} 0.1 + 0.4 + 0.45 + 0.05 \\ 0.2 + 0.2 + 0.3 + 0.15 \\ 0.3 + 0.6 + 0.15 + 0.1 \end{bmatrix} + \begin{bmatrix} 0.1 \\ 0.2 \\ 0.1 \end{bmatrix} \tag{46}$$

$$= \begin{bmatrix} 1.0 \\ 0.85 \\ 1.15 \end{bmatrix} + \begin{bmatrix} 0.1 \\ 0.2 \\ 0.1 \end{bmatrix} = \begin{bmatrix} 1.1 \\ 1.05 \\ 1.25 \end{bmatrix} \tag{47}$$

Step 2: Apply ReLU activation

$$\mathbf{h} = \text{ReLU}(\mathbf{z}) = \max(0, \mathbf{z}) = \begin{bmatrix} 1.1 \\ 1.05 \\ 1.25 \end{bmatrix} \tag{48}$$

Interpretation:

- Non-square matrix $\mathbf{W} \in \mathbb{R}^{3\times4}$ reduces dimension from 4 to 3
- Each row of $\mathbf{W}$ computes a weighted sum of inputs
- Bias adds learnable offset
- ReLU introduces non-linearity

Training: Weight matrix $\mathbf{W}$ is learned via backpropagation to minimize loss.

2 Dot Product

2.1 Definition and Mathematical Foundation

Definition 2.1 (Dot Product (Inner Product))**.** The dot product of two vectors $\mathbf{a}, \mathbf{b} \in \mathbb{R}^n$ is a scalar defined as:

$$\mathbf{a} \cdot \mathbf{b} = \mathbf{a}^T \mathbf{b} = \sum_{i=1}^{n} a_i b_i = a_1 b_1 + a_2 b_2 + \cdots + a_n b_n \tag{49}$$

Notation: Also written as $\langle \mathbf{a}, \mathbf{b} \rangle$ or $(\mathbf{a}, \mathbf{b})$
Result: A single real number (scalar)

Theorem 2.1 (Properties of Dot Product)**.** For vectors $\mathbf{a}, \mathbf{b}, \mathbf{c} \in \mathbb{R}^n$ and scalar $k \in \mathbb{R}$:

1. **Commutative:** $\mathbf{a} \cdot \mathbf{b} = \mathbf{b} \cdot \mathbf{a}$
2. **Distributive:** $\mathbf{a} \cdot (\mathbf{b} + \mathbf{c}) = \mathbf{a} \cdot \mathbf{b} + \mathbf{a} \cdot \mathbf{c}$
3. **Scalar multiplication:** $(k\mathbf{a}) \cdot \mathbf{b} = k(\mathbf{a} \cdot \mathbf{b}) = \mathbf{a} \cdot (k\mathbf{b})$
4. **Positive definite:** $\mathbf{a} \cdot \mathbf{a} \geq 0$, with equality iff $\mathbf{a} = \mathbf{0}$
5. **Cauchy-Schwarz inequality:** $|\mathbf{a} \cdot \mathbf{b}| \leq \|\mathbf{a}\| \|\mathbf{b}\|$

2.2 Geometric Interpretation

Theorem 2.2 (Geometric Formula)**.** The dot product can be expressed using vector magnitudes and the angle between them:

$$\mathbf{a} \cdot \mathbf{b} = \|\mathbf{a}\| \|\mathbf{b}\| \cos\theta \tag{50}$$

where θ is the angle between vectors $\mathbf{a}$ and $\mathbf{b}$.
Consequences:

- $\theta = 0$: $\mathbf{a} \cdot \mathbf{b} = \|\mathbf{a}\| \|\mathbf{b}\|$ (parallel, same direction)
- $\theta = 90$: $\mathbf{a} \cdot \mathbf{b} = 0$ (perpendicular/orthogonal)
- $\theta = 180$: $\mathbf{a} \cdot \mathbf{b} = -\|\mathbf{a}\| \|\mathbf{b}\|$ (parallel, opposite direction)

Definition 2.2 (Vector Magnitude using Dot Product)**.** The magnitude (or norm) of a vector can be computed using dot product:

$$\|\mathbf{a}\| = \sqrt{\mathbf{a} \cdot \mathbf{a}} = \sqrt{\sum_{i=1}^{n} a_i^2} \tag{51}$$

This is also called the Euclidean norm or L2 norm.

Definition 2.3 (Angle Between Vectors)**.** The angle between two non-zero vectors can be found by rearranging the geometric formula:

$$\cos\theta = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|} \implies \theta = \arccos\left(\frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|}\right) \tag{52}$$

2.3 Computational Examples

Example 2.1 (Basic Dot Product Computation)**.** Compute the dot product of:

$$\mathbf{a} = \begin{bmatrix} 3 \\ 4 \\ 5 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 2 \\ -1 \\ 3 \end{bmatrix} \tag{53}$$

Step 1: Apply formula

$$\mathbf{a} \cdot \mathbf{b} = \sum_{i=1}^{3} a_i b_i \tag{54}$$
$$= (3)(2) + (4)(-1) + (5)(3) \tag{55}$$
$$= 6 - 4 + 15 \tag{56}$$
$$= 17 \tag{57}$$

Step 2: Matrix notation verification

$$\mathbf{a}^T \mathbf{b} = \begin{bmatrix} 3 & 4 & 5 \end{bmatrix} \begin{bmatrix} 2 \\ -1 \\ 3 \end{bmatrix} = 17\checkmark \tag{58}$$

Result: The dot product is 17 (a positive scalar).

Example 2.2 (Computing Angle Between Vectors)**.** Find the angle between:

$$\mathbf{a} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 1 \\ 1 \end{bmatrix} \tag{59}$$

Step 1: Compute dot product

$$\mathbf{a} \cdot \mathbf{b} = (1)(1) + (0)(1) = 1 \tag{60}$$

Step 2: Compute magnitudes

$$\|\mathbf{a}\| = \sqrt{1^2 + 0^2} = 1 \tag{61}$$
$$\|\mathbf{b}\| = \sqrt{1^2 + 1^2} = \sqrt{2} \tag{62}$$

Step 3: Apply angle formula

$$\cos\theta = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\|\|\mathbf{b}\|} = \frac{1}{1 \cdot \sqrt{2}} = \frac{1}{\sqrt{2}} = \frac{\sqrt{2}}{2} \tag{63}$$

Step 4: Find angle

$$\theta = \arccos\left(\frac{\sqrt{2}}{2}\right) = 45 = \frac{\pi}{4} \text{ radians} \tag{64}$$

Interpretation: The vectors form a 45° angle, which makes geometric sense: $\mathbf{a}$ points along x-axis, $\mathbf{b}$ points diagonally.

Example 2.3 (Testing Orthogonality)**.** Check if these vectors are orthogonal:

$$\mathbf{a} = \begin{bmatrix} 3 \\ -2 \\ 1 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 2 \\ 3 \\ 0 \end{bmatrix} \tag{65}$$

Computation:

$$\mathbf{a} \cdot \mathbf{b} = (3)(2) + (-2)(3) + (1)(0) = 6 - 6 + 0 = 0 \tag{66}$$

Conclusion: Since $\mathbf{a} \cdot \mathbf{b} = 0$, the vectors are orthogonal (perpendicular).
This means $\cos\theta = 0 \implies \theta = 90$.

Example 2.4 (Projection of Vector onto Another)**.** Project vector $\mathbf{b}$ onto vector $\mathbf{a}$:

$$\mathbf{a} = \begin{bmatrix} 3 \\ 4 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 2 \\ 1 \end{bmatrix} \tag{67}$$

The projection formula is:

$$\text{proj}_{\mathbf{a}}\mathbf{b} = \frac{\mathbf{a} \cdot \mathbf{b}}{\mathbf{a} \cdot \mathbf{a}}\mathbf{a} \tag{68}$$

Step 1: Compute $\mathbf{a} \cdot \mathbf{b}$

$$\mathbf{a} \cdot \mathbf{b} = (3)(2) + (4)(1) = 6 + 4 = 10 \tag{69}$$

Step 2: Compute $\mathbf{a} \cdot \mathbf{a}$

$$\mathbf{a} \cdot \mathbf{a} = 3^2 + 4^2 = 9 + 16 = 25 \tag{70}$$

Step 3: Calculate projection

$$\text{proj}_{\mathbf{a}}\mathbf{b} = \frac{10}{25}\begin{bmatrix} 3 \\ 4 \end{bmatrix} = \frac{2}{5}\begin{bmatrix} 3 \\ 4 \end{bmatrix} = \begin{bmatrix} 1.2 \\ 1.6 \end{bmatrix} \tag{71}$$

Interpretation: The projection is the component of $\mathbf{b}$ along the direction of $\mathbf{a}$.
Verification - projection is parallel to a:

$$\begin{bmatrix} 1.2 \\ 1.6 \end{bmatrix} = 0.4 \begin{bmatrix} 3 \\ 4 \end{bmatrix} \checkmark \tag{72}$$

2.4 Applications in Machine Learning and Deep Learning

Critical ML/DL Applications of Dot Product

1. Similarity Measures:
Cosine similarity between vectors:

$$\text{similarity}(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\|\|\mathbf{b}\|} = \cos\theta \tag{73}$$

Range: [-1, 1] where 1 = identical direction, 0 = orthogonal, -1 = opposite
Applications:

- Document similarity in NLP

- Image similarity in computer vision
- Recommendation systems
- Clustering algorithms

2. Neural Network Layers:
Every neuron computes a weighted sum (dot product):

$$z = \mathbf{w}^T\mathbf{x} + b = \sum_{i=1}^{n} w_i x_i + b \tag{74}$$

Then applies activation: $a = \sigma(z)$
3. Attention Mechanisms (Transformers):
Attention scores are computed using dot products:

$$\text{score}(\mathbf{q}, \mathbf{k}) = \frac{\mathbf{q} \cdot \mathbf{k}}{\sqrt{d_k}} \tag{75}$$

where $\mathbf{q}$ = query, $\mathbf{k}$ = key, d_k = dimension
4. Linear Regression:
Prediction is a dot product:

$$\hat{y} = \mathbf{w}^T\mathbf{x} = w_1x_1 + w_2x_2 + \cdots + w_dx_d \tag{76}$$

5. Support Vector Machines (SVMs):
Decision boundary is based on dot product:

$$f(\mathbf{x}) = \text{sign}(\mathbf{w}^T\mathbf{x} + b) \tag{77}$$

Kernel trick: $K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i) \cdot \phi(\mathbf{x}_j)$
6. Distance Metrics:
Euclidean distance uses dot product:

$$\|\mathbf{a} - \mathbf{b}\|^2 = (\mathbf{a} - \mathbf{b}) \cdot (\mathbf{a} - \mathbf{b}) = \mathbf{a} \cdot \mathbf{a} - 2\mathbf{a} \cdot \mathbf{b} + \mathbf{b} \cdot \mathbf{b} \tag{78}$$

7. Principal Component Analysis:
Projection onto principal components:

$$z_i = \mathbf{v}_i^T\mathbf{x} \quad \text{(dot product with eigenvector)} \tag{79}$$

8. Gradient Descent:
Gradient is computed using dot products in backpropagation:

$$\frac{\partial L}{\partial w_i} = \sum_j \frac{\partial L}{\partial z_j}\frac{\partial z_j}{\partial w_i} = \sum_j \delta_j x_i \tag{80}$$

Example 2.5 (ML Application: Cosine Similarity for Document Comparison)**.** Two

documents represented as TF-IDF vectors:

$$\mathbf{d}_1 = [0.5, 0.3, 0.0, 0.8, 0.0] \quad \text{(Document 1)} \tag{81}$$
$$\mathbf{d}_2 = [0.4, 0.2, 0.1, 0.7, 0.0] \quad \text{(Document 2)} \tag{82}$$

Step 1: Compute dot product

$$\mathbf{d}_1 \cdot \mathbf{d}_2 = (0.5)(0.4) + (0.3)(0.2) + (0.0)(0.1) + (0.8)(0.7) + (0.0)(0.0) \tag{83}$$
$$= 0.20 + 0.06 + 0 + 0.56 + 0 \tag{84}$$
$$= 0.82 \tag{85}$$

Step 2: Compute magnitudes

$$\|\mathbf{d}_1\| = \sqrt{0.5^2 + 0.3^2 + 0^2 + 0.8^2 + 0^2} = \sqrt{0.98} \approx 0.990 \tag{86}$$
$$\|\mathbf{d}_2\| = \sqrt{0.4^2 + 0.2^2 + 0.1^2 + 0.7^2 + 0^2} = \sqrt{0.70} \approx 0.837 \tag{87}$$

Step 3: Compute cosine similarity

$$\text{similarity} = \frac{\mathbf{d}_1 \cdot \mathbf{d}_2}{\|\mathbf{d}_1\|\|\mathbf{d}_2\|} = \frac{0.82}{0.990 \times 0.837} \approx \frac{0.82}{0.829} \approx 0.989 \tag{88}$$

Interpretation:

- Similarity = 0.989 (very close to 1)
- Documents are very similar in content
- Would be good candidates for clustering together

Decision threshold: If similarity ¿ 0.8, consider documents as duplicates or highly related.

Example 2.6 (Deep Learning Application: Neuron Computation)**.** A single neuron with 4 inputs:

Weights: $\mathbf{w} = [0.5, -0.3, 0.8, 0.2]^T$ **Bias:** $b = 0.1$ **Input:** $\mathbf{x} = [1.0, 2.0, 0.5, 1.5]^T$

Step 1: Compute weighted sum (dot product)

$$z = \mathbf{w}^T\mathbf{x} + b \tag{89}$$
$$= (0.5)(1.0) + (-0.3)(2.0) + (0.8)(0.5) + (0.2)(1.5) + 0.1 \tag{90}$$
$$= 0.5 - 0.6 + 0.4 + 0.3 + 0.1 \tag{91}$$
$$= 0.7 \tag{92}$$

Step 2: Apply activation function (sigmoid)

$$a = \sigma(z) = \frac{1}{1 + e^{-z}} = \frac{1}{1 + e^{-0.7}} \approx \frac{1}{1.503} \approx 0.668 \tag{93}$$

Interpretation:

- Neuron receives 4 inputs
- Computes weighted sum via dot product

- Adds bias term
- Applies non-linear activation
- Outputs 0.668 (activation level)

In a neural network:

- This operation happens billions of times during training
- Efficient dot product computation is critical for performance
- GPUs excel at parallel dot product operations

Example 2.7 (Transformer Attention: Scaled Dot-Product Attention)**.** Simplified attention mechanism:

Query: $\mathbf{q} = [1.0, 0.5, 0.8]^T$ **Keys:**

$$\mathbf{K} = \begin{bmatrix} 1.2 & 0.3 & 0.7 \\ 0.8 & 0.9 & 0.5 \\ 0.5 & 0.4 & 1.0 \end{bmatrix} \tag{94}$$

(Each row is a key vector)

Values:

$$\mathbf{V} = \begin{bmatrix} 2.0 & 1.5 \\ 1.8 & 1.2 \\ 1.6 & 2.0 \end{bmatrix} \tag{95}$$

Step 1: Compute attention scores (dot products)

$$\text{score}_1 = \mathbf{q} \cdot \mathbf{k}_1 = (1.0)(1.2) + (0.5)(0.3) + (0.8)(0.7) = 1.91 \tag{96}$$
$$\text{score}_2 = \mathbf{q} \cdot \mathbf{k}_2 = (1.0)(0.8) + (0.5)(0.9) + (0.8)(0.5) = 1.65 \tag{97}$$
$$\text{score}_3 = \mathbf{q} \cdot \mathbf{k}_3 = (1.0)(0.5) + (0.5)(0.4) + (0.8)(1.0) = 1.50 \tag{98}$$

Step 2: Scale by $\sqrt{d_k}$

$$d_k = 3, \quad \sqrt{d_k} = \sqrt{3} \approx 1.732 \tag{99}$$

$$\text{scaled}_1 = 1.91/1.732 \approx 1.103 \tag{100}$$
$$\text{scaled}_2 = 1.65/1.732 \approx 0.953 \tag{101}$$
$$\text{scaled}_3 = 1.50/1.732 \approx 0.866 \tag{102}$$

Step 3: Apply softmax

$$e^{1.103} \approx 3.01, \quad e^{0.953} \approx 2.59, \quad e^{0.866} \approx 2.38 \tag{103}$$

$$\text{sum} = 3.01 + 2.59 + 2.38 = 7.98 \tag{104}$$

$$\alpha_1 = 3.01/7.98 \approx 0.377 \tag{105}$$
$$\alpha_2 = 2.59/7.98 \approx 0.325 \tag{106}$$
$$\alpha_3 = 2.38/7.98 \approx 0.298 \tag{107}$$

Step 4: Weighted sum of values

$$\text{output} = \alpha_1 \mathbf{v}_1 + \alpha_2 \mathbf{v}_2 + \alpha_3 \mathbf{v}_3 \tag{108}$$

$$= 0.377 \begin{bmatrix} 2.0 \\ 1.5 \end{bmatrix} + 0.325 \begin{bmatrix} 1.8 \\ 1.2 \end{bmatrix} + 0.298 \begin{bmatrix} 1.6 \\ 2.0 \end{bmatrix} \tag{109}$$

$$= \begin{bmatrix} 0.754 \\ 0.566 \end{bmatrix} + \begin{bmatrix} 0.585 \\ 0.390 \end{bmatrix} + \begin{bmatrix} 0.477 \\ 0.596 \end{bmatrix} \tag{110}$$

$$= \begin{bmatrix} 1.816 \\ 1.552 \end{bmatrix} \tag{111}$$

Interpretation:

- Query attends most to key 1 (highest score)
- Attention weights sum to 1
- Output is weighted combination of values
- Dot products enable efficient similarity computation

3 Cross Product

3.1 Definition and Mathematical Foundation

Definition 3.1 (Cross Product (Vector Product))**.** The cross product of two vectors $\mathbf{a}, \mathbf{b} \in \mathbb{R}^3$ is a vector $\mathbf{c} \in \mathbb{R}^3$ defined as:

$$\mathbf{a} \times \mathbf{b} = \begin{bmatrix} a_2b_3 - a_3b_2 \\ a_3b_1 - a_1b_3 \\ a_1b_2 - a_2b_1 \end{bmatrix} \tag{112}$$

Key characteristic: The result is a vector (unlike dot product which gives a scalar).
Restriction: Only defined for 3-dimensional vectors (or can be extended to 7D, but not commonly used).

Definition 3.2 (Determinant Notation)**.** The cross product can be expressed as a determinant:

$$\mathbf{a} \times \mathbf{b} = \det \begin{bmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ a_1 & a_2 & a_3 \\ b_1 & b_2 & b_3 \end{bmatrix} \tag{113}$$

where $\mathbf{i}, \mathbf{j}, \mathbf{k}$ are the standard unit vectors.
Expansion:

$$\mathbf{a} \times \mathbf{b} = (a_2b_3 - a_3b_2)\mathbf{i} - (a_1b_3 - a_3b_1)\mathbf{j} + (a_1b_2 - a_2b_1)\mathbf{k} \tag{114}$$

Theorem 3.1 (Properties of Cross Product)**.** For vectors $\mathbf{a}, \mathbf{b}, \mathbf{c} \in \mathbb{R}^3$ and scalar $k \in \mathbb{R}$:

1. **Anti-commutative:** $\mathbf{a} \times \mathbf{b} = -(\mathbf{b} \times \mathbf{a})$
2. **Not associative:** $(\mathbf{a} \times \mathbf{b}) \times \mathbf{c} \neq \mathbf{a} \times (\mathbf{b} \times \mathbf{c})$ (in general)
3. **Distributive:** $\mathbf{a} \times (\mathbf{b} + \mathbf{c}) = \mathbf{a} \times \mathbf{b} + \mathbf{a} \times \mathbf{c}$
4. **Scalar multiplication:** $(k\mathbf{a}) \times \mathbf{b} = k(\mathbf{a} \times \mathbf{b}) = \mathbf{a} \times (k\mathbf{b})$
5. **Orthogonality:** $\mathbf{a} \times \mathbf{b}$ is perpendicular to both $\mathbf{a}$ and $\mathbf{b}$:

$$(\mathbf{a} \times \mathbf{b}) \cdot \mathbf{a} = 0, \quad (\mathbf{a} \times \mathbf{b}) \cdot \mathbf{b} = 0 \tag{115}$$

6. **Zero product:** $\mathbf{a} \times \mathbf{b} = \mathbf{0} \iff \mathbf{a}$ and $\mathbf{b}$ are parallel
7. **Self cross product:** $\mathbf{a} \times \mathbf{a} = \mathbf{0}$

3.2 Geometric Interpretation

Theorem 3.2 (Geometric Formula)**.** The magnitude of the cross product is:

$$\|\mathbf{a} \times \mathbf{b}\| = \|\mathbf{a}\|\|\mathbf{b}\| \sin\theta \tag{116}$$

where θ is the angle between $\mathbf{a}$ and $\mathbf{b}$.
Geometric meaning: $\|\mathbf{a} \times \mathbf{b}\|$ equals the area of the parallelogram formed by $\mathbf{a}$ and $\mathbf{b}$.

Theorem 3.3 (Right-Hand Rule)**.** The direction of $\mathbf{a}\times\mathbf{b}$ is determined by the right-hand rule:

- Point fingers in direction of $\mathbf{a}$
- Curl fingers toward $\mathbf{b}$
- Thumb points in direction of $\mathbf{a}\times\mathbf{b}$

The resulting vector is perpendicular to the plane containing $\mathbf{a}$ and $\mathbf{b}$.

3.3 Computational Examples

Example 3.1 (Basic Cross Product Computation)**.** Compute the cross product:

$$\mathbf{a}=\begin{bmatrix}2\\3\\4\end{bmatrix},\quad \mathbf{b}=\begin{bmatrix}5\\6\\7\end{bmatrix} \tag{117}$$

Method 1: Component formula

$$\mathbf{a}\times\mathbf{b}=\begin{bmatrix}a_2b_3-a_3b_2\\a_3b_1-a_1b_3\\a_1b_2-a_2b_1\end{bmatrix} \tag{118}$$

$$=\begin{bmatrix}(3)(7)-(4)(6)\\(4)(5)-(2)(7)\\(2)(6)-(3)(5)\end{bmatrix} \tag{119}$$

$$=\begin{bmatrix}21-24\\20-14\\12-15\end{bmatrix}=\begin{bmatrix}-3\\6\\-3\end{bmatrix} \tag{120}$$

Method 2: Determinant expansion

$$\mathbf{a}\times\mathbf{b}=\det\begin{bmatrix}\mathbf{i}&\mathbf{j}&\mathbf{k}\\2&3&4\\5&6&7\end{bmatrix} \tag{121}$$

$$=\mathbf{i}\det\begin{bmatrix}3&4\\6&7\end{bmatrix}-\mathbf{j}\det\begin{bmatrix}2&4\\5&7\end{bmatrix}+\mathbf{k}\det\begin{bmatrix}2&3\\5&6\end{bmatrix} \tag{122}$$

$$=\mathbf{i}(21-24)-\mathbf{j}(14-20)+\mathbf{k}(12-15) \tag{123}$$

$$=-3\mathbf{i}+6\mathbf{j}-3\mathbf{k}=\begin{bmatrix}-3\\6\\-3\end{bmatrix} \tag{124}$$

Verification of orthogonality:

$$(\mathbf{a}\times\mathbf{b})\cdot\mathbf{a}=(-3)(2)+(6)(3)+(-3)(4)=-6+18-12=0\checkmark \tag{125}$$

$$(\mathbf{a}\times\mathbf{b})\cdot\mathbf{b}=(-3)(5)+(6)(6)+(-3)(7)=-15+36-21=0\checkmark \tag{126}$$

The cross product is indeed perpendicular to both original vectors!

Example 3.2 (Computing Area of Parallelogram)**.** Find the area of the parallelogram formed by:

$$\mathbf{a} = \begin{bmatrix} 3 \\ 0 \\ 0 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 0 \\ 4 \\ 0 \end{bmatrix} \tag{127}$$

Step 1: Compute cross product

$$\mathbf{a} \times \mathbf{b} = \begin{bmatrix} (0)(0) - (0)(4) \\ (0)(0) - (3)(0) \\ (3)(4) - (0)(0) \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 12 \end{bmatrix} \tag{128}$$

Step 2: Compute magnitude

$$\|\mathbf{a} \times \mathbf{b}\| = \sqrt{0^2 + 0^2 + 12^2} = 12 \tag{129}$$

Area: The area of the parallelogram is 12 square units.
Verification using base × height:

- Base = $\|\mathbf{a}\| = 3$
- Height = $\|\mathbf{b}\| = 4$
- Since vectors are perpendicular ($\theta = 90$): Area = $3 \times 4 = 12$

Example 3.3 (Testing Parallel Vectors)**.** Check if vectors are parallel:

$$\mathbf{a} = \begin{bmatrix} 2 \\ 4 \\ 6 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} \tag{130}$$

Observation: $\mathbf{a} = 2\mathbf{b}$, so vectors are parallel.
Compute cross product:

$$\mathbf{a} \times \mathbf{b} = \begin{bmatrix} (4)(3) - (6)(2) \\ (6)(1) - (2)(3) \\ (2)(2) - (4)(1) \end{bmatrix} \tag{131}$$

$$= \begin{bmatrix} 12 - 12 \\ 6 - 6 \\ 4 - 4 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \tag{132}$$

Conclusion: Cross product is zero vector, confirming vectors are parallel.

Example 3.4 (Standard Unit Vector Cross Products)**. Standard basis vectors:**

$$\mathbf{i} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \quad \mathbf{j} = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, \quad \mathbf{k} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \tag{133}$$

Cyclic pattern (right-hand rule):

$$\mathbf{i} \times \mathbf{j} = \mathbf{k} \tag{134}$$

$$\mathbf{j} \times \mathbf{k} = \mathbf{i} \tag{135}$$

$$\mathbf{k} \times \mathbf{i} = \mathbf{j} \tag{136}$$

Anti-cyclic pattern:

$$\mathbf{j} \times \mathbf{i} = -\mathbf{k} \tag{137}$$
$$\mathbf{k} \times \mathbf{j} = -\mathbf{i} \tag{138}$$
$$\mathbf{i} \times \mathbf{k} = -\mathbf{j} \tag{139}$$

Self cross products:

$$\mathbf{i} \times \mathbf{i} = \mathbf{j} \times \mathbf{j} = \mathbf{k} \times \mathbf{k} = \mathbf{0} \tag{140}$$

Verification - Example: $\mathbf{i} \times \mathbf{j}$

$$\begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \times \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} (0)(0) - (0)(1) \\ (0)(0) - (1)(0) \\ (1)(1) - (0)(0) \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} = \mathbf{k}\checkmark \tag{141}$$

3.4 Applications in Machine Learning and Computer Vision

ML/DL Applications of Cross Product

1. Computer Graphics and 3D Vision:
Surface Normal Computation: Given two edges of a triangle:

$$\mathbf{n} = \mathbf{e}_1 \times \mathbf{e}_2 \tag{142}$$

The cross product gives the normal vector perpendicular to the surface.
Applications:

- Lighting calculations (Phong shading)
- Back-face culling in rendering
- Surface reconstruction
- 3D mesh processing

2. Robotics and Pose Estimation:
Computing rotational velocities and angular momentum:

$$\boldsymbol{\omega} = \mathbf{r} \times \mathbf{v} \tag{143}$$

Used in:

- Robot kinematics
- Camera pose estimation
- SLAM (Simultaneous Localization and Mapping)

3. Physics-Based Neural Networks:
Modeling physical constraints:

- Torque calculations: $\boldsymbol{\tau} = \mathbf{r} \times \mathbf{F}$
- Angular momentum conservation

- Rigid body dynamics

4. Point Cloud Processing:
Computing surface normals for 3D point clouds:

$$\mathbf{n}_p = (\mathbf{p}_2 - \mathbf{p}_1) \times (\mathbf{p}_3 - \mathbf{p}_1) \tag{144}$$

Used in:

- 3D object detection
- Scene understanding
- Autonomous driving

5. Geometric Deep Learning:

- Graph neural networks with spatial features
- Mesh convolutional networks
- Point cloud networks (PointNet++)

6. Camera Calibration:
Essential matrix and fundamental matrix computations involve cross products for epipolar geometry.
7. Volume Computation:
Scalar triple product (involves cross product):

$$V = |\mathbf{a} \cdot (\mathbf{b} \times \mathbf{c})| \tag{145}$$

Volume of parallelepiped formed by three vectors.

Example 3.5 (Computer Vision: Computing Surface Normal)**.** A triangular mesh face with vertices:

$$\mathbf{p}_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \tag{146}$$

$$\mathbf{p}_2 = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \tag{147}$$

$$\mathbf{p}_3 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \tag{148}$$

Step 1: Compute edge vectors

$$\mathbf{e}_1 = \mathbf{p}_2 - \mathbf{p}_1 = \begin{bmatrix} -1 \\ 1 \\ 0 \end{bmatrix} \tag{149}$$

$$\mathbf{e}_2 = \mathbf{p}_3 - \mathbf{p}_1 = \begin{bmatrix} -1 \\ 0 \\ 1 \end{bmatrix} \tag{150}$$

Step 2: Compute cross product (normal vector)

$$\mathbf{n} = \mathbf{e}_1 \times \mathbf{e}_2 \tag{151}$$

$$= \begin{bmatrix} (1)(1) - (0)(0) \\ (0)(-1) - (-1)(1) \\ (-1)(0) - (1)(-1) \end{bmatrix} \tag{152}$$

$$= \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} \tag{153}$$

Step 3: Normalize to unit normal

$$\|\mathbf{n}\| = \sqrt{1^2 + 1^2 + 1^2} = \sqrt{3} \tag{154}$$

$$\hat{\mathbf{n}} = \frac{\mathbf{n}}{\|\mathbf{n}\|} = \frac{1}{\sqrt{3}} \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 0.577 \\ 0.577 \\ 0.577 \end{bmatrix} \tag{155}$$

Application in lighting:
Lambertian shading intensity:

$$I = k_d(\hat{\mathbf{n}} \cdot \hat{\mathbf{l}}) \tag{156}$$

where $\hat{\mathbf{l}}$ is the light direction and k_d is the diffuse coefficient.
If light direction is $\hat{\mathbf{l}} = [0, 0, 1]^T$:

$$I = k_d(0.577 \times 0 + 0.577 \times 0 + 0.577 \times 1) = 0.577 k_d \tag{157}$$

This determines how bright the surface appears based on its orientation to the light.

Example 3.6 (Robotics: Computing Torque)**.** A robot arm with:

- Position vector from joint to force application: $\mathbf{r} = [0.3, 0.2, 0]^T$ meters
- Applied force: $\mathbf{F} = [0, 0, -10]^T$ Newtons (downward gravity)

Compute torque:

$$\boldsymbol{\tau} = \mathbf{r} \times \mathbf{F} \tag{158}$$

$$= \begin{bmatrix} 0.3 \\ 0.2 \\ 0 \end{bmatrix} \times \begin{bmatrix} 0 \\ 0 \\ -10 \end{bmatrix} \tag{159}$$

$$= \begin{bmatrix} (0.2)(-10) - (0)(0) \\ (0)(0) - (0.3)(-10) \\ (0.3)(0) - (0.2)(0) \end{bmatrix} \tag{160}$$

$$= \begin{bmatrix} -2 \\ 3 \\ 0 \end{bmatrix} \text{ N·m} \tag{161}$$

Interpretation:

- Torque magnitude: $\|\boldsymbol{\tau}\| = \sqrt{4+9} = \sqrt{13} \approx 3.61$ N·m
- Direction: perpendicular to both $\mathbf{r}$ and $\mathbf{F}$
- This torque causes rotation about the joint
- Robot controller must compensate to maintain position

In ML for robotics:

- Neural networks learn to predict required torques
- Physics-informed networks include cross product constraints
- Reinforcement learning optimizes control policies

Example 3.7 (3D Computer Vision: Volume Computation)**.** Compute volume of tetrahedron with vertices at origin and:

$$\mathbf{a} = \begin{bmatrix} 2 \\ 0 \\ 0 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 0 \\ 3 \\ 0 \end{bmatrix}, \quad \mathbf{c} = \begin{bmatrix} 0 \\ 0 \\ 4 \end{bmatrix} \tag{162}$$

Formula: Volume $= \frac{1}{6}|\mathbf{a} \cdot (\mathbf{b} \times \mathbf{c})|$
Step 1: Compute cross product

$$\mathbf{b} \times \mathbf{c} = \begin{bmatrix} (3)(4) - (0)(0) \\ (0)(0) - (0)(4) \\ (0)(0) - (3)(0) \end{bmatrix} = \begin{bmatrix} 12 \\ 0 \\ 0 \end{bmatrix} \tag{163}$$

Step 2: Compute scalar triple product

$$\mathbf{a} \cdot (\mathbf{b} \times \mathbf{c}) = \begin{bmatrix} 2 \\ 0 \\ 0 \end{bmatrix} \cdot \begin{bmatrix} 12 \\ 0 \\ 0 \end{bmatrix} = 24 \tag{164}$$

Step 3: Compute volume

$$V = \frac{1}{6}|24| = 4 \text{ cubic units} \tag{165}$$

Verification: For a tetrahedron with perpendicular edges along axes:

$$V = \frac{1}{6} \times \text{base area} \times \text{height} = \frac{1}{6} \times (2 \times 3) \times 4 = 4\checkmark \tag{166}$$

Application:

- Mesh volume computation in 3D reconstruction
- Collision detection in physics simulations
- Shape analysis in medical imaging

3.5 Cross Product vs Dot Product Summary

3.6 Why Cross Product is Less Common in ML

Property	**Dot Product**	**Cross Product**
Result	Scalar	Vector
Dimension	Any dimension	Only 3D (primarily)
Formula	$\mathbf{a} \cdot \mathbf{b} = \sum a_i b_i$	$\mathbf{a} \times \mathbf{b}$ = perpendicular vector
Geometric	$\|\mathbf{a}\|\|\mathbf{b}\| \cos\theta$	$\|\mathbf{a}\|\|\mathbf{b}\| \sin\theta$
Commutative?	Yes: $\mathbf{a} \cdot \mathbf{b} = \mathbf{b} \cdot \mathbf{a}$	No: $\mathbf{a} \times \mathbf{b} = -\mathbf{b} \times \mathbf{a}$
Measures	Projection, similarity	Area, perpendicularity
Zero when	Perpendicular ($\theta = 90$)	Parallel ($\theta = 0$ or 180)
ML Uses	Similarity, neurons, attention	Surface normals, 3D geometry

Table 1: Comparison of Dot Product and Cross Product

Limitations of Cross Product in ML

1. **Dimension Restriction:**
   - Only defined for 3D vectors (primarily)
   - ML often works with high-dimensional data (100s to 1000s of dimensions)
   - Dot product generalizes to any dimension
2. **Computational Efficiency:**
   - Dot product: $O(n)$ operations, highly parallelizable
   - Cross product: More complex, produces vector output
   - Neural networks prefer scalar outputs from dot products
3. **Application Domains:**
   - Dot product: Similarity, projections, transformations (universal)
   - Cross product: Geometric operations, 3D spatial reasoning (specialized)
4. **Where Cross Product Appears:**
   - Computer graphics and rendering
   - 3D computer vision (surface normals, pose estimation)
   - Robotics (torque, angular momentum)
   - Physics-informed neural networks
   - Point cloud processing
   - Geometric deep learning on 3D data

4 Conclusion

This document covered three essential topics for machine learning and deep learning:

4.1 Key Takeaways

1. **Linear Transformation of Non-square Matrices:**
   - Common in ML due to dimension mismatches
   - Used in dimensionality reduction (PCA)
   - Neural network layers often have different input/output sizes
   - Understanding null space reveals information loss
   - Pseudo-inverse handles non-invertibility
2. **Dot Product:**
   - Fundamental operation in all ML/DL algorithms
   - Measures similarity and projection
   - Core of neural network computations
   - Used in attention mechanisms (Transformers)
   - Enables efficient parallel computation
   - Works in any dimension
3. **Cross Product:**
   - Specialized for 3D geometric operations
   - Essential in computer vision and graphics
   - Computes surface normals and areas
   - Used in robotics for torque calculations
   - Less common in general ML, but crucial for 3D applications
   - Produces vectors perpendicular to input vectors

4.2 Practical Recommendations

Best Practices

For Implementation:

- Use NumPy/PyTorch for efficient dot product: `np.dot()`, `@` operator
- Leverage GPU acceleration for matrix operations
- Use broadcasting to avoid explicit loops
- For cross product: `np.cross()` in NumPy
- Normalize vectors when computing similarity (cosine similarity)

For ML Applications:

- Dot product: similarity search, neural networks, recommender systems
- Cross product: 3D vision, surface reconstruction, robotics
- Non-square transformations: dimensionality reduction, feature engineering
- Always check dimensions before matrix operations
- Use pseudo-inverse for non-square systems

4.3 Further Study

To deepen understanding:

- Study tensor operations for modern deep learning
- Learn attention mechanisms in detail (Transformers)
- Explore 3D deep learning (PointNet, Mesh R-CNN)
- Understand matrix calculus for backpropagation
- Practice implementing algorithms from scratch
- Study geometric deep learning frameworks

4.4 Connection to Previous Topics

These concepts build upon:

- **Matrix inverse:** Pseudo-inverse for non-square matrices
- **Rank:** Determines dimension of transformed space
- **Null space:** Information lost in non-square transformations
- **Column space:** Range of transformation
- **Change of basis:** PCA as change to principal component basis

5 Assessment Questions

5.1 Level 1: Basic Understanding

1. **What is a non-square matrix?** Provide two examples from machine learning.
2. **Give the definition** of dot product and cross product. What is the key difference in their outputs?
3. **What is the difference between** a left inverse and a right inverse? When does each exist?
4. **Calculate**: Compute the dot product of $\mathbf{a} = \begin{bmatrix} 2 \\ 3 \\ 1 \end{bmatrix}$ and $\mathbf{b} = \begin{bmatrix} 1 \\ -1 \\ 2 \end{bmatrix}$.
5. **If** $\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 0 & 1 \\ 1 & 0 \end{bmatrix}$ and $\mathbf{x} = \begin{bmatrix} 2 \\ 3 \end{bmatrix}$, find $\mathbf{Ax}$.
6. **True or False**: The dot product of two vectors is always a scalar.
7. **Identify whether each operation** produces a scalar or vector:
   (a) Dot product: $\mathbf{a} \cdot \mathbf{b}$
   (b) Cross product: $\mathbf{a} \times \mathbf{b}$
   (c) Matrix-vector product: $\mathbf{Ax}$
8. **Compute**: $\mathbf{a} \times \mathbf{b}$ where $\mathbf{a} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$ and $\mathbf{b} = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$.
9. **What are the two fundamental properties** that define a linear transformation?
10. **Why is the cross product** only defined for 3D vectors (primarily)?

5.2 Level 2: Intermediate

11. **What is the role of dot product** in neural network layers? Provide the formula.
12. **Compute** the left inverse of $\mathbf{A} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}$.
13. **State the geometric formula** for dot product and explain its meaning.
14. **Given** $\mathbf{a} = \begin{bmatrix} 3 \\ 4 \end{bmatrix}$ and $\mathbf{b} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$, compute the projection of $\mathbf{b}$ onto $\mathbf{a}$.
15. **What is the null space** of a non-square matrix? Give geometric interpretation.

16. **Given transformation** matrix $\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 0 & 1 \\ 1 & 0 \end{bmatrix}$, find the dimension of its column space and null space.
17. **Calculate**: Find the angle between $\mathbf{a} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ and $\mathbf{b} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$ using dot product.
18. **If** $\mathbf{a} = \begin{bmatrix} 2 \\ 3 \\ 4 \end{bmatrix}$ and $\mathbf{b} = \begin{bmatrix} 5 \\ 6 \\ 7 \end{bmatrix}$, compute $\mathbf{a} \times \mathbf{b}$.
19. **Solve**: For $\mathbf{A} \in \mathbb{R}^{3\times 2}$ with full column rank, find the left inverse formula.
20. **Calculate** the area of parallelogram formed by $\mathbf{a} = \begin{bmatrix} 3 \\ 0 \\ 0 \end{bmatrix}$ and $\mathbf{b} = \begin{bmatrix} 0 \\ 4 \\ 0 \end{bmatrix}$.

5.3 Level 3: Advanced

21. **Prove** that the dot product is commutative: $\mathbf{a} \cdot \mathbf{b} = \mathbf{b} \cdot \mathbf{a}$.
22. **Why does a non-square matrix** $\mathbf{A} \in \mathbb{R}^{m\times n}$ with $m > n$ have a left inverse but not a right inverse?
23. **Explain the role of dot product** in attention mechanisms (Transformers). Provide the scaled dot-product attention formula.
24. **Compare the computational complexity** of computing:
    (a) Dot product of two n-dimensional vectors
    (b) Cross product of two 3D vectors
    (c) Matrix-vector product $\mathbf{Ax}$ where $\mathbf{A} \in \mathbb{R}^{m\times n}$
25. **Design a PCA transformation** using non-square matrix to reduce 1000-dimensional data to 50 dimensions. Specify the transformation matrix dimensions.
26. **Explain how cross product** is used in computing surface normals for 3D computer vision. Provide the formula.
27. **What happens to the rank** when you apply a non-square transformation $\mathbf{A} \in \mathbb{R}^{m\times n}$ where $m < n$? Prove your answer.
28. **Prove** that the cross product is anti-commutative: $\mathbf{a} \times \mathbf{b} = -(\mathbf{b} \times \mathbf{a})$.
29. **Explain the relationship between** null space dimension and information loss in dimensionality reduction.
30. **How do you determine** if a non-square matrix has full column rank or full row rank? What are the implications for ML applications?

5.4 Multiple Choice Questions

31. The dot product of two vectors $\mathbf{a}, \mathbf{b} \in \mathbb{R}^n$ is:
    (a) A vector in $\mathbb{R}^n$
    (b) A scalar
    (c) A matrix in $\mathbb{R}^{n \times n}$
    (d) A tensor

32. For a non-square matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ with $m > n$ and full column rank:
    (a) Only right inverse exists
    (b) Only left inverse exists
    (c) Both inverses exist
    (d) No inverse exists

33. The cross product $\mathbf{a} \times \mathbf{b}$ is zero if and only if:
    (a) $\mathbf{a}$ and $\mathbf{b}$ are perpendicular
    (b) $\mathbf{a}$ and $\mathbf{b}$ are parallel
    (c) $\mathbf{a}$ or $\mathbf{b}$ is zero
    (d) Both (b) and (c)

34. In a neural network layer with input dimension 784 and output dimension 128, the weight matrix $\mathbf{W}$ has dimensions:
    (a) 784×128
    (b) 128×784
    (c) 784×784
    (d) 128×128

35. The computational complexity of dot product for two n-dimensional vectors is:
    (a) $O(n^2)$
    (b) $O(n)$
    (c) $O(1)$
    (d) $O(\log n)$

5.5 Programming Exercises

P1. Implement dot product computation in Python without using NumPy's built-in functions.

P2. Write a function to compute the left inverse of a non-square matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ where $m > n$.

P3. Create a function to compute cross product of two 3D vectors and verify orthogonality.

P4. Implement cosine similarity between two document vectors using dot product.

P5. Build a simple PCA dimensionality reduction function that uses non-square transformation matrix.

6 Solutions to Assessment Questions

6.1 Level 1 Solutions

A1. A non-square matrix is a matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ where $m \neq n$.

ML Examples:

1. *Neural Network Layer*: Weight matrix $\mathbf{W} \in \mathbb{R}^{128 \times 784}$ maps 784-dimensional input to 128-dimensional output.

2. *PCA Transformation*: Projection matrix $\mathbf{W} \in \mathbb{R}^{50 \times 1000}$ reduces 1000-dimensional features to 50 dimensions.

Explanation: Non-square matrices are common in ML because input and output dimensions often differ.

A2. Dot Product: $\mathbf{a} \cdot \mathbf{b} = \sum_{i=1}^{n} a_i b_i = \mathbf{a}^T \mathbf{b}$ (produces a scalar)

Cross Product: $\mathbf{a} \times \mathbf{b}$ produces a vector perpendicular to both $\mathbf{a}$ and $\mathbf{b}$ (only for 3D vectors)

Key difference: Dot product gives a scalar; cross product gives a vector.

A3. Left Inverse: For $\mathbf{A} \in \mathbb{R}^{m \times n}$ with $m > n$ and full column rank, $\mathbf{A}_L^{-1} = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T$ where $\mathbf{A}_L^{-1} \mathbf{A} = \mathbf{I}_n$.

Right Inverse: For $\mathbf{A} \in \mathbb{R}^{m \times n}$ with $m < n$ and full row rank, $\mathbf{A}_R^{-1} = \mathbf{A}^T (\mathbf{A} \mathbf{A}^T)^{-1}$ where $\mathbf{A} \mathbf{A}_R^{-1} = \mathbf{I}_m$.

Explanation: Left inverse exists when there are more rows than columns; right inverse exists when there are more columns than rows.

A4.

$$\begin{aligned} \mathbf{a} \cdot \mathbf{b} &= (2)(1) + (3)(-1) + (1)(2) && (167) \\ &= 2 - 3 + 2 && (168) \\ &= 1 && (169) \end{aligned}$$

A5.

$$\begin{aligned} \mathbf{A}\mathbf{x} &= \begin{bmatrix} 1 & 2 \\ 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 2 \\ 3 \end{bmatrix} && (170) \\ &= \begin{bmatrix} 1(2) + 2(3) \\ 0(2) + 1(3) \\ 1(2) + 0(3) \end{bmatrix} && (171) \\ &= \begin{bmatrix} 8 \\ 3 \\ 2 \end{bmatrix} && (172) \end{aligned}$$

A6. True. The dot product always produces a single scalar value.

Explanation: $\mathbf{a} \cdot \mathbf{b} = \sum_{i=1}^{n} a_i b_i$ is a sum of products, resulting in one number.

A7.

(a) Dot product: **Scalar**

(b) Cross product: **Vector**

(c) Matrix-vector product: **Vector**

A8.

$$\mathbf{a} \times \mathbf{b} = \begin{bmatrix}1\\0\\0\end{bmatrix} \times \begin{bmatrix}0\\1\\0\end{bmatrix} \tag{173}$$

$$= \begin{bmatrix}(0)(0)-(0)(1)\\(0)(0)-(1)(0)\\(1)(1)-(0)(0)\end{bmatrix} \tag{174}$$

$$= \begin{bmatrix}0\\0\\1\end{bmatrix} = \mathbf{k} \tag{175}$$

Explanation: This follows the right-hand rule: $\mathbf{i} \times \mathbf{j} = \mathbf{k}$.

A9. The two fundamental properties are:

1. **Additivity**: $T(\mathbf{u} + \mathbf{v}) = T(\mathbf{u}) + T(\mathbf{v})$
2. **Homogeneity**: $T(c\mathbf{u}) = cT(\mathbf{u})$ for any scalar c

A10. The cross product is primarily defined for 3D vectors because:

- The formula requires exactly 3 components to compute the perpendicular vector
- The right-hand rule applies specifically to 3D space
- While it can be extended to 7D, it's not commonly used in higher dimensions
- Most geometric applications (surface normals, torque) are in 3D space

6.2 Level 2 Solutions

A11. In neural network layers, dot product computes the weighted sum:

$$z = \mathbf{w}^T\mathbf{x} + b = \sum_{i=1}^{n} w_i x_i + b$$

Explanation: Each neuron computes a dot product between input vector and weight vector, then adds bias and applies activation function.

A12. For $\mathbf{A} = \begin{bmatrix}1 & 0\\0 & 1\\1 & 1\end{bmatrix}$:

Step 1: Compute $\mathbf{A}^T\mathbf{A}$

$$\mathbf{A}^T\mathbf{A} = \begin{bmatrix}1 & 0 & 1\\0 & 1 & 1\end{bmatrix}\begin{bmatrix}1 & 0\\0 & 1\\1 & 1\end{bmatrix} = \begin{bmatrix}2 & 1\\1 & 2\end{bmatrix}$$

Step 2: Invert $\mathbf{A}^T\mathbf{A}$

$$(\mathbf{A}^T\mathbf{A})^{-1} = \frac{1}{3}\begin{bmatrix}2 & -1\\-1 & 2\end{bmatrix}$$

Step 3: Compute left inverse

$$\mathbf{A}_L^{-1} = \frac{1}{3}\begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix}\begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix} = \frac{1}{3}\begin{bmatrix} 2 & -1 & 1 \\ -1 & 2 & 1 \end{bmatrix}$$

A13. Geometric formula: $\mathbf{a} \cdot \mathbf{b} = \|\mathbf{a}\|\|\mathbf{b}\| \cos\theta$

Meaning: The dot product equals the product of vector magnitudes times the cosine of the angle between them. This relates algebraic computation to geometric relationships.

A14. Projection formula: $\text{proj}_{\mathbf{a}}\mathbf{b} = \frac{\mathbf{a}\cdot\mathbf{b}}{\mathbf{a}\cdot\mathbf{a}}\mathbf{a}$

Step 1: $\mathbf{a} \cdot \mathbf{b} = (3)(2) + (4)(1) = 10$

Step 2: $\mathbf{a} \cdot \mathbf{a} = 3^2 + 4^2 = 25$

Step 3: $\text{proj}_{\mathbf{a}}\mathbf{b} = \frac{10}{25}\begin{bmatrix} 3 \\ 4 \end{bmatrix} = \begin{bmatrix} 1.2 \\ 1.6 \end{bmatrix}$

A15. The **null space** of matrix $\mathbf{A}$ is:

$$\mathcal{N}(\mathbf{A}) = \{\mathbf{x} : \mathbf{A}\mathbf{x} = \mathbf{0}\}$$

Geometric interpretation: All vectors that get mapped to zero by the transformation. For non-square matrices with $m < n$, the null space is non-trivial, meaning information is lost.

Example: In dimensionality reduction, vectors in the null space direction are collapsed to zero.

A16. For $\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 0 & 1 \\ 1 & 0 \end{bmatrix} \in \mathbb{R}^{3\times 2}$:

- $\text{rank}(\mathbf{A}) = 2$ (columns are linearly independent)
- $\dim(\mathcal{C}(\mathbf{A})) = 2$ (column space dimension equals rank)
- $\dim(\mathcal{N}(\mathbf{A})) = 2 - 2 = 0$ (null space is trivial since $m > n$)

A17. Using dot product formula:

$$\mathbf{a} \cdot \mathbf{b} = (1)(1) + (0)(1) = 1 \tag{176}$$

$$\|\mathbf{a}\| = 1, \quad \|\mathbf{b}\| = \sqrt{2} \tag{177}$$

$$\cos\theta = \frac{1}{1 \cdot \sqrt{2}} = \frac{1}{\sqrt{2}} \tag{178}$$

$$\theta = \arccos\left(\frac{1}{\sqrt{2}}\right) = 45 = \frac{\pi}{4} \tag{179}$$

A18.

$$\mathbf{a} \times \mathbf{b} = \begin{bmatrix} (3)(7) - (4)(6) \\ (4)(5) - (2)(7) \\ (2)(6) - (3)(5) \end{bmatrix} \tag{180}$$

$$= \begin{bmatrix} 21 - 24 \\ 20 - 14 \\ 12 - 15 \end{bmatrix} = \begin{bmatrix} -3 \\ 6 \\ -3 \end{bmatrix} \tag{181}$$

A19. For $\mathbf{A} \in \mathbb{R}^{m\times n}$ with $m > n$ and full column rank:

Left inverse formula:

$$\mathbf{A}_L^{-1} = (\mathbf{A}^T\mathbf{A})^{-1}\mathbf{A}^T$$

Verification: $\mathbf{A}_L^{-1}\mathbf{A} = (\mathbf{A}^T\mathbf{A})^{-1}\mathbf{A}^T\mathbf{A} = \mathbf{I}_n$ ✓

A20. Area of parallelogram $= \|\mathbf{a} \times \mathbf{b}\|$

$$\mathbf{a} \times \mathbf{b} = \begin{bmatrix}(0)(0) - (0)(4)\\(0)(0) - (3)(0)\\(3)(4) - (0)(0)\end{bmatrix} = \begin{bmatrix}0\\0\\12\end{bmatrix} \tag{182}$$

$$\|\mathbf{a} \times \mathbf{b}\| = 12 \tag{183}$$

Verification: Since vectors are perpendicular, area $= 3 \times 4 = 12$ ✓

6.3 Level 3 Solutions

A21. Proof of commutativity:

Let $\mathbf{a} = [a_1, a_2, \ldots, a_n]^T$ and $\mathbf{b} = [b_1, b_2, \ldots, b_n]^T$.

$$\mathbf{a} \cdot \mathbf{b} = \sum_{i=1}^{n} a_i b_i \tag{184}$$

$$= \sum_{i=1}^{n} b_i a_i \quad \text{(multiplication is commutative)} \tag{185}$$

$$= \mathbf{b} \cdot \mathbf{a} \tag{186}$$

Therefore, $\mathbf{a} \cdot \mathbf{b} = \mathbf{b} \cdot \mathbf{a}$ ✓

A22. For $\mathbf{A} \in \mathbb{R}^{m \times n}$ with $m > n$:

- **Left inverse exists** if $\text{rank}(\mathbf{A}) = n$ (full column rank)
- **Right inverse cannot exist** because:
  - Right inverse requires $\mathbf{A}\mathbf{A}_R^{-1} = \mathbf{I}_m$
  - But $\mathbf{A}$ has only n columns, so $\mathbf{A}\mathbf{A}_R^{-1}$ has at most rank $n < m$
  - Cannot equal $\mathbf{I}_m$ which has rank m

Geometric interpretation: With more rows than columns, the transformation maps to a lower-dimensional space, so we can recover the input (left inverse) but cannot create a full identity on the output space.

A23. In attention mechanisms, dot product computes similarity scores:

Scaled Dot-Product Attention:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V}$$

Key role:

- $\mathbf{Q}\mathbf{K}^T$ computes dot products between queries and keys

- Higher dot product = higher similarity = more attention
- Scaling by $\sqrt{d_k}$ prevents extreme values
- Softmax converts scores to probability distribution

A24. Computational complexity:
(a) Dot product: $O(n)$ operations (n multiplications + n-1 additions)
(b) Cross product: $O(1)$ operations (fixed 9 operations for 3D vectors)
(c) Matrix-vector product: $O(mn)$ operations (m dot products, each O(n))
Summary: Dot product is linear, cross product is constant (for 3D), matrix-vector product is quadratic in dimensions.

A25. PCA Transformation Design:
Transformation: $\mathbf{Z} = \mathbf{XW}$ where:

- $\mathbf{X} \in \mathbb{R}^{N \times 1000}$ (N samples, 1000 features)
- $\mathbf{W} \in \mathbb{R}^{1000 \times 50}$ (non-square transformation matrix)
- $\mathbf{Z} \in \mathbb{R}^{N \times 50}$ (reduced dimension)

Steps:

1. Center data: $\mathbf{X}_{centered} = \mathbf{X} - \boldsymbol{\mu}$
2. Compute covariance: $\mathbf{C} = \frac{1}{N-1}\mathbf{X}_{centered}^T \mathbf{X}_{centered}$
3. Eigendecomposition: $\mathbf{C} = \mathbf{V}\boldsymbol{\Lambda}\mathbf{V}^T$
4. Select top 50 eigenvectors: $\mathbf{W} = \mathbf{V}_{:,:50}$ (first 50 columns)
5. Project: $\mathbf{Z} = \mathbf{X}_{centered}\mathbf{W}$

A26. Surface Normal Computation:
For triangular mesh with vertices $\mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_3$:

$$\mathbf{e}_1 = \mathbf{p}_2 - \mathbf{p}_1 \tag{187}$$
$$\mathbf{e}_2 = \mathbf{p}_3 - \mathbf{p}_1 \tag{188}$$
$$\mathbf{n} = \mathbf{e}_1 \times \mathbf{e}_2 \quad \text{(normal vector)} \tag{189}$$
$$\hat{\mathbf{n}} = \frac{\mathbf{n}}{\|\mathbf{n}\|} \quad \text{(unit normal)} \tag{190}$$

Application: Used in lighting calculations, back-face culling, and 3D reconstruction.

A27. Rank under non-square transformation:
For $\mathbf{A} \in \mathbb{R}^{m \times n}$ with $m < n$:
Theorem: $\text{rank}(\mathbf{A}) \leq \min(m, n) = m$
Proof:

- Rank equals dimension of column space
- Column space is subspace of $\mathbb{R}^m$
- Maximum dimension of subspace in $\mathbb{R}^m$ is m

- Therefore, $\text{rank}(\mathbf{A}) \leq m$

Implication: When $m < n$, we cannot have full column rank, so information is lost (null space is non-trivial).

A28. Proof of anti-commutativity:

$$\mathbf{a} \times \mathbf{b} = \begin{bmatrix} a_2b_3 - a_3b_2 \\ a_3b_1 - a_1b_3 \\ a_1b_2 - a_2b_1 \end{bmatrix} \tag{191}$$

$$\mathbf{b} \times \mathbf{a} = \begin{bmatrix} b_2a_3 - b_3a_2 \\ b_3a_1 - b_1a_3 \\ b_1a_2 - b_2a_1 \end{bmatrix} \tag{192}$$

$$= \begin{bmatrix} -(a_2b_3 - a_3b_2) \\ -(a_3b_1 - a_1b_3) \\ -(a_1b_2 - a_2b_1) \end{bmatrix} \tag{193}$$

$$= -(\mathbf{a} \times \mathbf{b}) \tag{194}$$

Therefore, $\mathbf{a} \times \mathbf{b} = -(\mathbf{b} \times \mathbf{a})$ ✓

A29. Null space and information loss:

Relationship: $\dim(\mathcal{N}(\mathbf{A})) = n - \text{rank}(\mathbf{A})$

In dimensionality reduction:

- Larger null space $\implies$ more information lost
- Vectors in null space are mapped to zero
- Only information in column space is preserved
- Rank-nullity theorem: $\dim(\mathcal{C}(\mathbf{A})) + \dim(\mathcal{N}(\mathbf{A})) = n$

Example: PCA with $\mathbf{W} \in \mathbb{R}^{1000\times 50}$:

- Null space dimension $= 1000 - 50 = 950$
- 950 dimensions of information are lost
- Only 50 principal directions are preserved

A30. Determining rank properties:

For $\mathbf{A} \in \mathbb{R}^{m\times n}$:

Full column rank ($\text{rank}(\mathbf{A}) = n$):

- All columns are linearly independent
- Null space is trivial: $\dim(\mathcal{N}(\mathbf{A})) = 0$
- Left inverse exists if $m > n$
- No information loss in transformation

Full row rank ($\text{rank}(\mathbf{A}) = m$):

- All rows are linearly independent

- Column space spans entire $\mathbb{R}^m$
- Right inverse exists if $m < n$
- Transformation is surjective

ML implications:

- Full column rank: Good for feature extraction (no redundancy)
- Full row rank: Good for covering output space completely
- Rank deficiency: Indicates redundant features or insufficient model capacity

6.4 Multiple Choice Solutions

A31. (b) A scalar

Explanation: The dot product $\mathbf{a} \cdot \mathbf{b} = \sum_{i=1}^{n} a_i b_i$ is a sum of products, resulting in a single real number (scalar).

A32. (b) Only left inverse exists

Explanation: For $m > n$ with full column rank, $\mathbf{A}^T\mathbf{A}$ is invertible, allowing left inverse. Right inverse cannot exist because $\mathbf{A}$ cannot span full $\mathbb{R}^m$ space.

A33. (d) Both (b) and (c)

Explanation: Cross product is zero when vectors are parallel ($\mathbf{a} = k\mathbf{b}$) OR when either vector is zero. Both conditions make $\sin\theta = 0$.

A34. (b) 128×784

Explanation: For transformation $\mathbf{h} = \mathbf{W}\mathbf{x} + \mathbf{b}$ where $\mathbf{x} \in \mathbb{R}^{784}$ and $\mathbf{h} \in \mathbb{R}^{128}$, we need $\mathbf{W} \in \mathbb{R}^{128\times784}$ to satisfy dimension compatibility: $(128 \times 784)(784 \times 1) = (128 \times 1)$.

A35. (b) $O(n)$

Explanation: Dot product requires n multiplications and $n-1$ additions, giving $O(n)$ time complexity. This is linear in the vector dimension.

7 Practice Problems

7.1 Computational Problems

1. **Calculate**: Compute dot product of $\mathbf{a} = [1, 2, 3, 4]^T$ and $\mathbf{b} = [2, -1, 0, 1]^T$.
2. **If** $\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 0 & 3 \\ 1 & 2 \end{bmatrix}$, find $\mathbf{Ax}$ where $\mathbf{x} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$.
3. **Compute** cross product: $\mathbf{a} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$ and $\mathbf{b} = \begin{bmatrix} 4 \\ 5 \\ 6 \end{bmatrix}$.
4. **Calculate** the angle between $\mathbf{a} = \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}$ and $\mathbf{b} = \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix}$.
5. **Find** the projection of $\mathbf{b} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$ onto $\mathbf{a} = \begin{bmatrix} 3 \\ 4 \end{bmatrix}$.

7.2 Conceptual Problems

6. **Why can't** a non-square matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ with $m \neq n$ have a true inverse?
7. **How does** the dot product relate to cosine similarity in machine learning?
8. **Explain** why the cross product is perpendicular to both input vectors.
9. **Why does** PCA use a non-square transformation matrix?
10. **How do** dot products enable efficient computation in neural networks?

7.3 Application Problems

11. **Design** a neural network layer that transforms 256-dimensional input to 128-dimensional output. Specify the weight matrix dimensions.
12. **Implement** cosine similarity computation between two document vectors.
13. **Create** a function to compute surface normal for a triangular face in 3D space.
14. **Calculate** the left inverse for a given non-square matrix with full column rank.
15. **Derive** the formula for projection of a vector onto another using dot product.

End of Notes

These comprehensive notes provide the mathematical foundation and practical applications needed for understanding modern machine learning and deep learning algorithms.